A template/ approach to Reproducible Generative Research

Research infrastructure that documents itself — a case study for meta-science and science-integrity teams



The problem

- Research groups publish claims about their own tools and pipelines that no one outside the team can re-check.
- Manuscripts describing infrastructure go stale the moment the infrastructure changes underneath them.
- Reproducibility efforts usually stop at the data and code layer — the documentation layer is still hand-maintained and drifts silently.



Landscape

Where meta-science tooling stands today

- Electronic lab notebooks and preregistration platforms address the experiment layer, not the infrastructure-description layer.
- Literate-programming tools (R Markdown, Jupyter, Quarto) bind prose to code within one document, but rarely to a whole repository's live state.
- Very few public examples exist of a manuscript that introspects and reports on the software system that produced it, at repository scale.



What template_template is

- A manuscript that is generated FROM the repository it describes, not written ABOUT it from memory.
- Every metric in the paper — module counts, test counts, pipeline stages — is computed live by template_template's own introspection code.
- One of 25 public exemplar templates in the same monorepo, each independently tested and coverage-gated.

Source: infrastructure/project/public_scope.py

“Autopoietic: the paper regenerates itself from the code it describes.”

— template_template/README.md

Source: projects/templates/template_template/README.md

How it works

Introspection

- src/template_template/core/introspection.py discovers infrastructure/ subpackages, the pipeline.yaml stage DAG, and the public exemplar roster.
- The scan reads the filesystem and YAML directly — it does not depend on any external service or cached snapshot.
- Function-level entry points (discover_infrastructure_modules, discover_projects, load_pipeline_stages_from_yaml) each return typed, testable dataclasses.



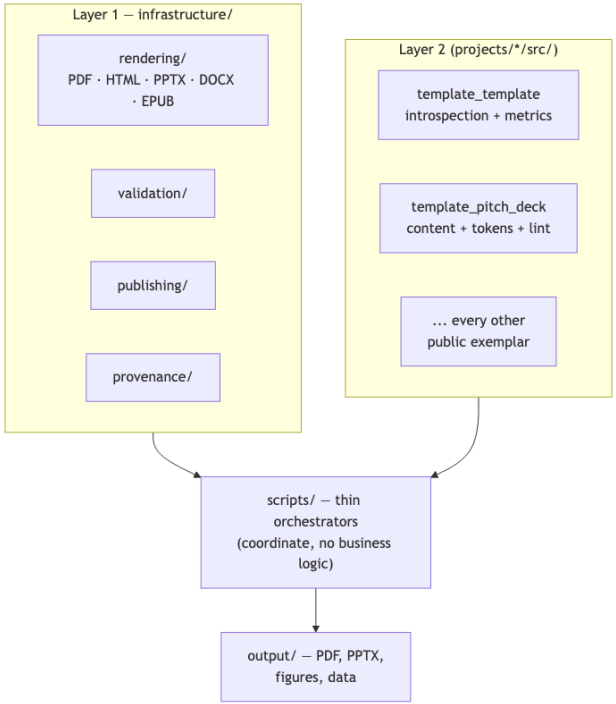
Metrics and injection

- metrics.py turns the introspection scan into a dictionary of ${variable} values — the same token convention used across every exemplar's manuscript.
- inject_metrics.py substitutes those values into manuscript/*.md, writing the resolved text to output/manuscript/ for rendering.
- Re-running the pipeline regenerates the paper from current repository state — the citation and the artifact cannot drift apart.

Visualization

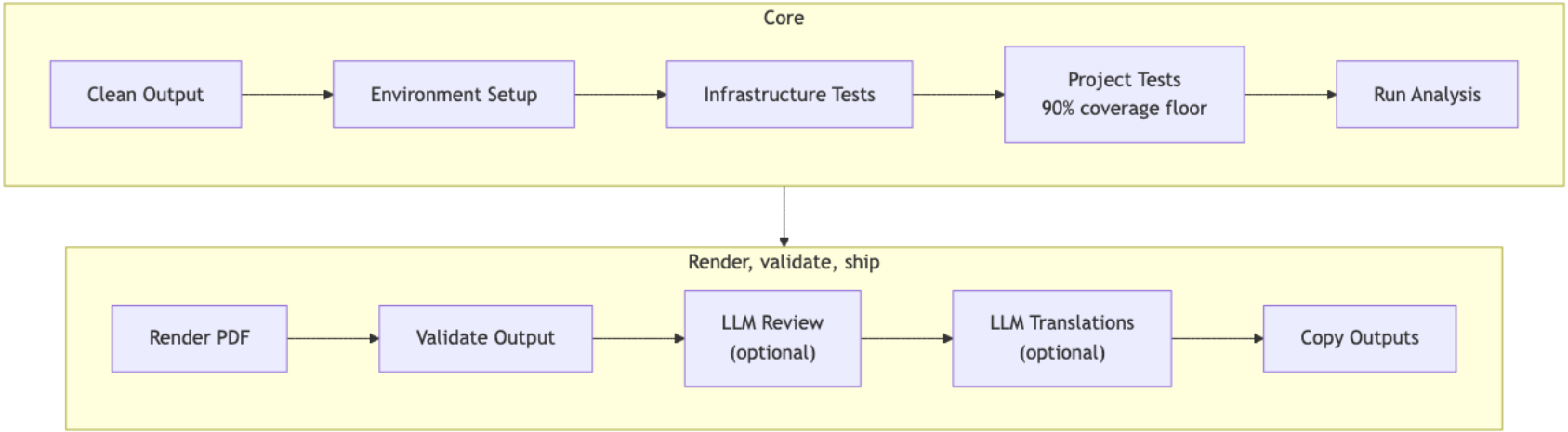
- Four figures — architecture overview, pipeline stages, module inventory, comparative feature matrix — are generated from the same live data, not hand-drawn.
- generate_architecture_viz.py orchestrates dedicated figure modules sharing one palette module, so the visual language stays consistent across all four.

Two-layer architecture, at a glance



Source: CLAUDE.md

The core pipeline, stage by stage



Source: CLAUDE.md

Proof, not adjectives



Measured, not asserted

161 tests

95.55% coverage on template_template's own source — the same gate every exemplar in the repo must pass.

Source: docs/_generated/COUNTS.md

No partial credit

- 90% project-coverage floor and a no-mocks testing policy apply to this project exactly as they do across all 25 exemplars.
- Every pipeline run either passes its own gate or fails loudly — there is no partial-credit reporting path.

Source: infrastructure/project/public_scope.py



Already public

- Concept DOI 10.5281/zenodo.20419007. Intended v2.0.0 is unreleased.
- Prior v1.0.9 has 9 historical references: Zenodo, GitHub, PyPI (sandbox), IPFS, Software Heritage, GitHub Pages, Netlify, Hugging Face, OSF.
- Licensed Apache-2.0 — the introspection code, not only the prose, is available for a science-integrity team to audit directly at docxology/template_template.

Source: projects/templates/template_template/manuscript/config.yaml



The pattern generalizes



The full roster, not one cherry-picked example

- projects/templates/ holds all 25 public exemplars: template_active_inference, template_advanced_literature_review, template_autopoiesis, template_autoresearch_project, template_autoscientists, template_code_project, template_data_descriptor, template_eda_notebook, template_experiment_tree, template_formal, template_gold_refinement, template_literature_meta_analysis, template_madlib, template_methods_paper, template_newspaper, template_pitch_deck, template_pools_rules_tools, template_prose_project, template_redacted_report, template_registered_report, template_search_project, template_sia, template_storybook, template_template, template_textbook.

Source: docs/_generated/active_projects.md



One contract across all public exemplars

- template_active_inference is called out here only as one concrete instance to point to — every name in that list is an independently-built exemplar following the identical two-layer architecture, not a hand-picked best case.
- Neither exemplar's coverage gate or drift check required any special-casing to add — the contract is uniform across all 25 projects, and the folder itself is the evidence, not a claim about it.

Source: docs/_generated/active_projects.md

Two-layer architecture

- Layer 1 (infrastructure/): generic, reusable — rendering, validation, publishing, provenance, testing utilities.
- Layer 2 (projects/{name}/src/): domain-specific — each exemplar's own algorithms, kept out of infrastructure/ by convention and enforced by drift checks.
- Thin orchestrator scripts (projects/{name}/scripts/) coordinate the two layers but implement no business logic themselves.



Governance and confidentiality

- Only projects/templates/ is public and CI-gated; private working projects render locally through the same pipeline without ever being tracked.
- A dedicated audit (scripts/audit/check_tracked_all.py) fails CI if any private project path is accidentally committed.



Scientific integrity, by construction

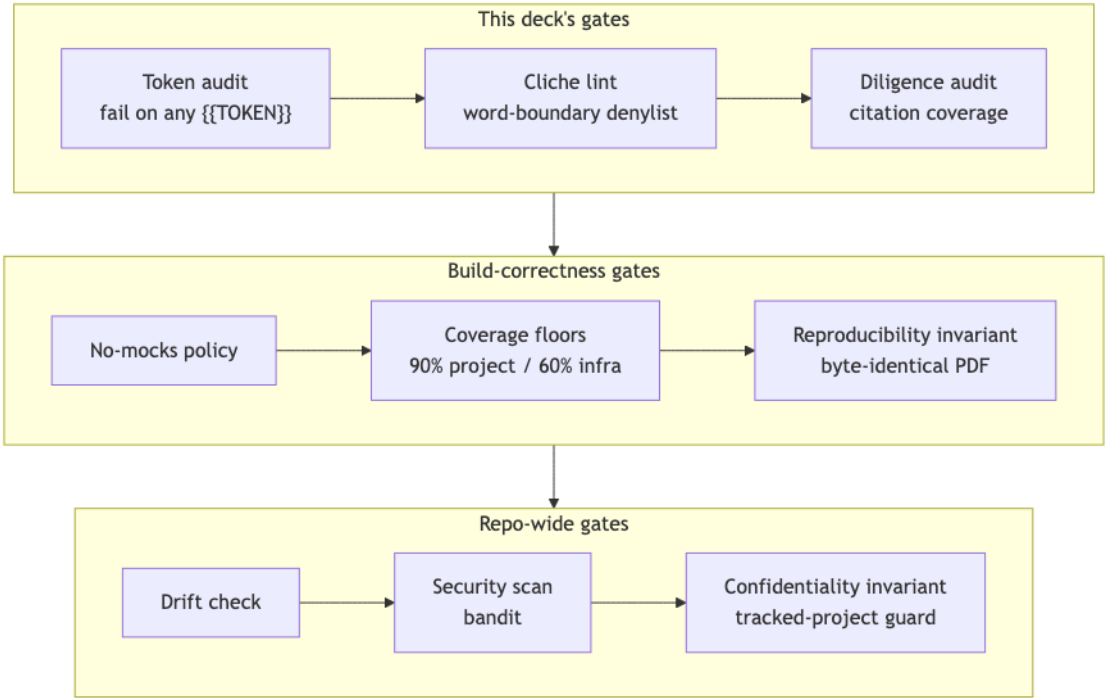
Independent gates, not one

- No-mocks testing policy, CI-enforced: scripts/audit/verify_no_mocks.py fails the build if any test imports MagicMock, mocker.patch, or unittest.mock — a checked gate, not a style preference.
- Coverage floors are enforced per project (90%) and for shared infrastructure (60%), not averaged away across the monorepo.
- A byte-level reproducibility invariant: two renders of identical content in the same environment produce an identical PDF — verified by a real, passing test, not assumed across every OS/toolchain combination.

Source: tests/infra_tests/rendering/test_slide_deck.py



The integrity stack, gate by gate



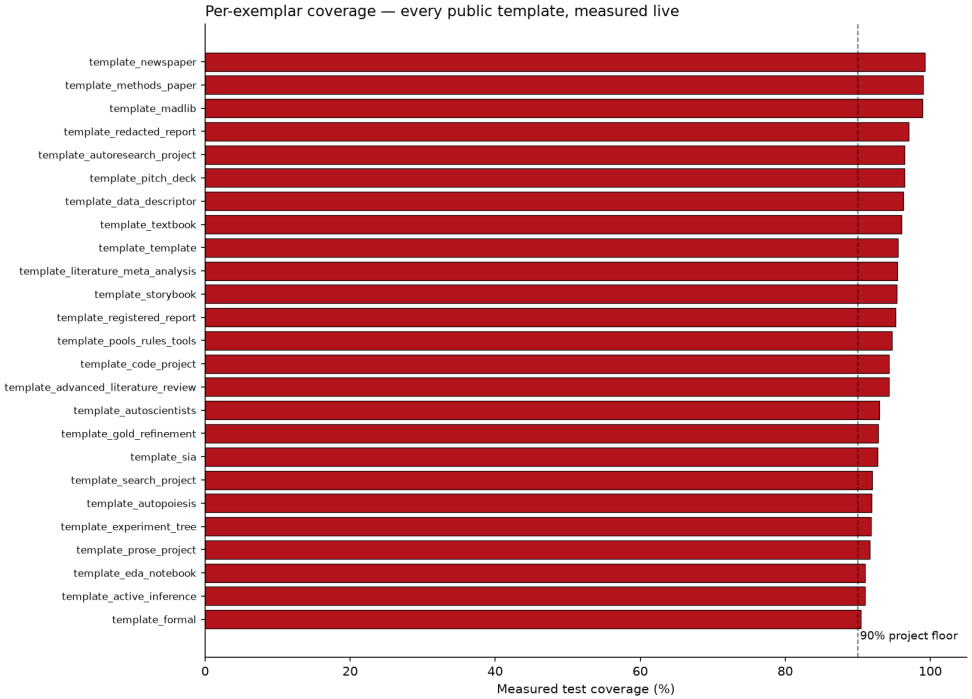
Source: projects/templates/template_pitch_deck/src/template_pitch_deck/token_resolution.py

Coverage isn't a headline number

- Every one of 25 public exemplars is measured independently, not summarized into one repo-wide percentage.
- The chart on the next slide is generated from the same docs/_generated/COUNTS.md factsheet this deck's own 95.55% figure comes from — one source of truth, two presentations.

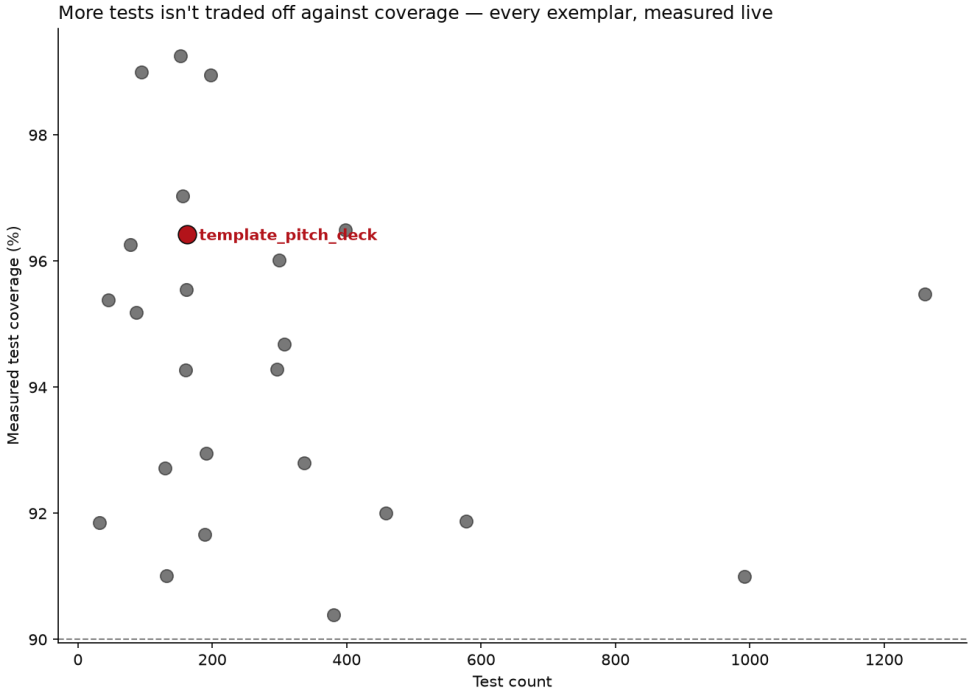
Source: docs/_generated/COUNTS.md

Per-exemplar coverage, measured live



Source: docs/_generated/COUNTS.md

Test volume and coverage aren't a tradeoff



Source: docs/_generated/COUNTS.md

This deck audits itself

- src/token_resolution.py fails the build on any unresolved double-curly-brace placeholder — the same discipline this slide's own template_template token was resolved by.
- src/cliche_lint.py and src/diligence_audit.py both run inside render_orchestration.py itself, before that deck length's own PDF/PPTX is written — a slide referencing a live-sourced token with no citation blocks that length's render, it does not just fail a separate, skippable check.
- None of these checks are generic infrastructure — they are this project's own src/, built specifically because a pitch deck is exactly the kind of document that tempts unverifiable claims.

Source: projects/templates/template_pitch_deck/src/template_pitch_deck/diligence_audit.py



The chain of custody: slide → QR → GitHub

- Every slide in this deck carries its own QR code, linking to a standalone, real Markdown page for that exact slide under output/slides_standalone/.
- Once this output/ directory is committed and pushed, a photo of a projected or printed slide can be traced back to its precise source page on GitHub — not just to the deck as a whole. Locally, the generated page already exists; the QR only resolves once it is actually published.
- Mechanically, this is the same source-citation system with one more hop: a citation links out to evidence; the QR links to the slide's own generated page, which repeats that same citation.

Source: projects/templates/template_pitch_deck/src/template_pitch_deck/standalone_slides.py

Cite this deck

- This pitch deck is its own exemplar in the same monorepo — it has its own src/, tests, and citable identity, distinct from the DOI of template_template, the project it pitches.
- This deck's own DOI: 10.5281/zenodo.21281509.
- That status is read live from this project's own manuscript/config.yaml at render time — the moment a real Zenodo deposit is recorded there, this slide's own text updates on the next regeneration, the same way every other fact in this deck does.

Source: projects/templates/template_pitch_deck/manuscript/config.yaml

What's actually inside infrastructure/



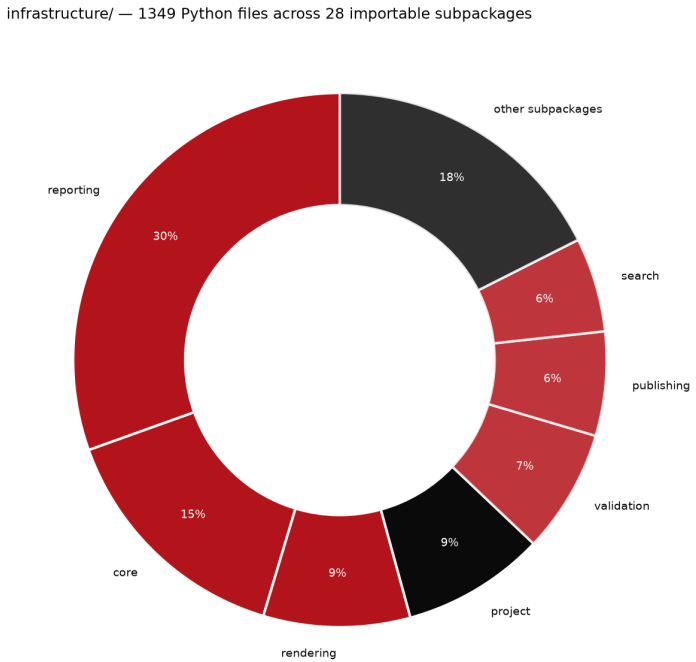
Not a black box

- 28 importable subpackages, 1352 tracked Python files total under infrastructure/ — a real inventory anyone can `git ls-files` themselves, not a claimed abstraction.
- The largest single subpackage, reporting, holds 411 files — still a minority of the whole, not a monolith wearing a modular label.
- Every number on this slide is computed live by src/infra_facts.py at render time, reusing the exact same counters docs/_generated/COUNTS.md is built from.
- The next chart narrows the total to importable subpackages, excluding 3 top-level Python file(s).

Source: projects/templates/template_pitch_deck/src/template_pitch_deck/infra_facts.py



infrastructure/, by subpackage



Source: projects/templates/template_pitch_deck/src/template_pitch_deck/infra_facts.py

Roadmap

What comes next for this exemplar

- Extend token-sourced facts beyond one pitch subject to any exemplar in the roster, so a new deck for a different project needs only new content YAML, not new code.
- Add a second validated pitch (a broader meta-science-group deck) reusing the same rendering and audit pipeline.
- Track deck-generation coverage and drift in the same CI gates that already watch every other public exemplar.



The ask

- Adopt this pattern for your own methods papers: bind every reported number to the code that produced it, not to a hand-typed table.
- Pilot it on one existing manuscript in your group before committing to a full rewrite.
- Happy to pair with your team through a first regeneration cycle and help define your own introspection surface, if useful — no prior engagements to point to yet, this pattern is newly forkable.



Funding & consulting

- The code itself is available to adopt directly today — real, tested, and licensed for reuse, no engagement required to start.
- Open to funding conversations for a dedicated integration (a new pitch subject, a distinct theme, a deeper introspection surface) — no such engagement exists yet, this is an invitation to scope one, not an established program.
- Open to scoping a consulting engagement on request — pairing through a first regeneration cycle, or adapting the two-layer architecture to an existing codebase outside this monorepo. No such engagement has happened yet; this is an invitation, not a track record.



Questions

Research infrastructure that documents itself — a case study for meta-science and science-integrity teams

